Measuring the Knowing-Doing Gap in LLM Agents with a Fair Bayesian Oracle

Sagar Deb

Abstract

LLM agents are increasingly evaluated on long-horizon tasks whose state is hidden: a demand shift, a failing supplier. The consistent finding is that performance degrades as horizons grow, and cost alone cannot say why, because an agent may have failed to infer what was happening, or may have inferred it correctly and failed to act (the knowing-doing gap). Existing measurements cannot separate the two: their reference policies see privileged information the agent does not. We introduce BENCHNAME, a 26-week supply-chain replenishment benchmark built as a factored POMDP with six hidden factor processes, designed so that a *fair* reference policy is computable: an exact Bayes filter per factor drives a rollout policy on the identical observation stream the agent receives. Scoring each run between a symptom-blind base-stock floor (0) and this oracle (1) gives a skill score, and each week's written rationale gives a detection lag and a knowing-doing rate, separating state estimation from control. On twenty seeds with controlled stress profiles, Claude Sonnet 5, GPT-5.4, and DeepSeek-V4-Pro detect 85–92% of hidden failures, typically within a week of onset; yet the two stronger models recover only 55% and 64% of the available cost headroom, DeepSeek-V4-Pro falls below the symptom-blind floor on 9 of 20 seeds, and where stress persists, 41–49% of correctly diagnosed stress weeks still end in stockout. The expensive step for current agents comes after diagnosis: converting a correct stated belief into costly action. The benchmark makes that step a measured quantity.

1 Introduction

LLM agents are increasingly evaluated on long-horizon decision tasks: running a vending machine for months [cite: VendingBench 2502.15840], managing a retail store [cite: RetailBench 2606.15862], operating supply chains [cite: AIMBench 2508.11416, InvAgent 2407.11384]. These tasks share a structure. The world has hidden state — a demand shift, a failing supplier — that the agent can only infer from noisy symptoms, and performance depends on acting on those inferences week after week. Benchmarks in this genre consistently report that agent performance degrades over long horizons. What they largely cannot report is *why*.

When an agent incurs excess cost on such a task, two distinct failure modes are possible. It may have failed to *infer* the hidden state, never recognizing that demand had shifted; or it may have inferred correctly and failed to *act*, stating the correct diagnosis in its reasoning while placing the wrong order. The second failure mode is documented across strategic games, tool use, and agentic tasks under the name of the **knowing-doing gap** [cite: BrokenLinks 2605.00226; 2605.14038; 2511.13240; 2504.16078]: models' stated beliefs are often correct while their actions lag or contradict them. In the language of partially observable decision processes, these are failures of *state estimation* and failures of *control*, and they call for different fixes — better inference scaffolding in the first case, better policy elicitation in the second. Telling them apart, however, requires knowing what a correct inference *was worth*.

Preprint.

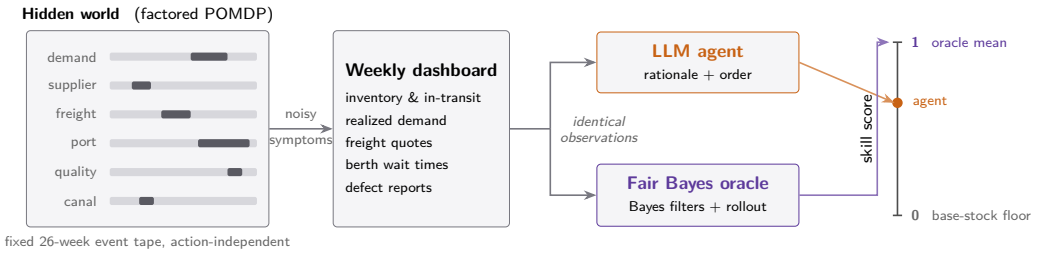

Figure 1: **Measuring the knowing-doing gap with a fair oracle.** Six hidden factor processes evolve on a fixed, action-independent event tape (left; dark segments mark stress regimes) and emit noisy symptoms into a weekly dashboard. The identical observation stream feeds both the LLM agent and a Bayes-filter reference policy (one exact filter per factor driving a rollout), so neither sees the hidden state. The agent's episode cost is placed on a skill scale anchored by a symptom-blind base-stock floor (0) and the fair oracle's 20-replication mean (1): shortfall on this scale is attributable to acting on beliefs, not to forming them.

Existing evidence for the knowing-doing gap does not support that accounting. It is either qualitative — manual trace inspection and post-hoc labeling [cite: BrokenLinks] — or quantified against a reference policy that sees privileged information: the oracle in RetailBench, for instance, is computed with access to ground truth the agent never observes [cite: RetailBench]. A privileged oracle conflates the two failure modes by construction, because the shortfall it measures bundles "did not know" and "knew but did not act" into one number. To attribute cost to the knowing-doing gap specifically, the reference policy must be denied exactly the information the agent is denied.

We introduce a benchmark in which that fair reference is computable. The task is a 26-week inventory-replenishment problem, formalized as a factored POMDP with six hidden factor processes whose states the agent must infer from noisy weekly symptoms (Section 3). Because the hidden structure is factored and the event tape fixed, an explicit Bayesian reference policy is feasible: an exact Bayes filter per factor driving a rollout policy, conditioned on exactly the observation stream the LLM receives. This reference is fair rather than privileged; Section 4 states the one disclosed asymmetry. Together with a symptom-blind base-stock floor, it yields a **skill score** locating each agent between ignoring the symptoms (0) and matching the fair Bayesian (1), and two rationale-based metrics, **detection lag** and the **knowing-doing rate**, that separate state estimation from control. Twenty seeds, grouped by stress profile (ISOLATED, PERSISTENT, COMPOUND), make the interaction of hidden failures a controlled variable.

Our contributions are:

1. A six-factor supply-chain POMDP benchmark with a fair, computable Bayes-filter oracle: an exact-filter rollout policy conditioned on the identical observation stream as the agent, referenced as the mean of 20 replications per seed. Fairness is what makes agent shortfall attributable to acting rather than confounded with knowing.

2. A controlled stress axis (ISOLATED / PERSISTENT / COMPOUND) that varies whether and how hidden failures overlap while holding the world model fixed.

3. Three metrics separating state estimation from control — skill score, detection lag, knowing-doing rate — evaluated over 60 episodes: three models (Claude Sonnet 5, GPT-5.4, DeepSeek-V4-Pro) on twenty seeds, under a rules-only prompt with no strategy playbook.

Across 60 episodes, detection proves nearly uniform: all three models name 85–92% of hidden-factor episodes, typically within a week of onset. Costs diverge sharply nonetheless, with one model scoring below the symptom-blind floor on 9 of 20 seeds despite detection as accurate as the others'. The gap also concentrates: on PERSISTENT seeds, 41–49% of correctly diagnosed stress weeks still end in stockout. What separates models is the distance between seeing a problem and acting on it, and that distance widens precisely where the environment punishes the intuitive response.

2 Related Work

Long-horizon agent benchmarks. A growing family of benchmarks runs LLM agents through extended business simulations: Vending-Bench [cite: 2502.15840] documents meltdown loops over months of vending-machine operation; RetailBench [cite: 2606.15862] reports incomplete evidence acquisition and no consistent long-horizon policy in supermarket management; CFO-bench [cite: 2603.23638] and YC-Bench [cite: 2604.01212] extend the genre to finance and startups. In supply chains specifically, AIM-Bench [cite: 2508.11416] isolates decision biases (mean anchoring, demand chasing, bullwhip amplification) in inventory tasks, and InvAgent [cite: 2407.11384] studies multi-echelon inventory control. These benchmarks report *that* performance degrades; their reference points — absolute profit, human baselines, or oracles with access to ground truth — do not separate failures of inference from failures of action. Where RetailBench scores against a privileged oracle, our reference policy is conditioned on the agent's own observation stream.

The knowing-doing gap. That stated beliefs and actions diverge in LLMs is established across settings: in strategic games, models articulate correct beliefs about opponents yet fail to act on them [cite: BrokenLinks 2605.00226]; models that correctly assess a tool is necessary still decline to use it [cite: 2605.14038]; stated confidence dissociates from chosen actions [cite: 2511.13240]; and agents act greedily against their own stated analysis [cite: 2504.16078]. This literature identifies the gap qualitatively or in single-step decisions. We complement it with a sequential, cost-weighted measurement: how much a correct belief was worth, week by week, against a Bayesian policy holding the same beliefs-relevant information.

Belief tracking in agents. Implicit beliefs degrade over long contexts without explicit representation [cite: BeliefMemory 2605.05583], and raw interaction history entangles spurious signals [cite: PABU 2602.09138]. Our detection-lag and knowing-doing metrics quantify exactly the boundary these works target — between maintaining a belief and using it — in a setting where the belief's economic value is computable.

3 BENCHNAME: Task and Environment

In week 2 of one benchmark episode, the dashboard of an electronics importer shows a supplier whose scorecard still reads on-time but whose realized fill was 53% — 8 of the 15 units ordered actually shipped. Claude Sonnet 5, playing the importer's replenishment manager, pays $25 for a supplier audit and gets back one sentence: "spot is currently slipping." It locks the freight rate for three weeks while the rate index is low, flies 20 units in to cover the near-term gap, and moves the week's order to a more reliable supplier. Every week of the benchmark has this shape: noisy symptoms in, one committed order plus optional paid levers out, and a written rationale stating what the agent believes is happening (Figure 1).

3.1 The weekly decision

The agent manages the importer's replenishment for a 26-week episode. Each week it reads the dashboard — on-hand inventory, orders in transit, realized demand, freight quotes, berth waiting times, canal transit counts, supplier scorecards, quality reports — and commits an order: a quantity, a route (through the Suez Canal, or slower and dearer around the Cape), and a supplier. Before committing it may pull any of four mitigation levers:

- **lock** the current freight rate for the coming weeks;
- **air-expedite** a limited number of units, which land the next week and bypass the port;
- **inspect** an arriving batch for defects before it lands;
- **buy information** — an audit or analyst briefing that reveals one factor's true state.

Costs accrue for units held, for demand missed, and for every lever pulled; the episode score is total cost, lower is better. The price ladder is the task's core tension: sea freight is $4 per unit and takes three weeks, air is $15, and every unit of missed demand costs $20. A manager who reads the symptoms early can buy its way out of trouble cheaply; one who reads them late cannot. The agent must also write a short free-text rationale each week; Section 4 derives two metrics from it.

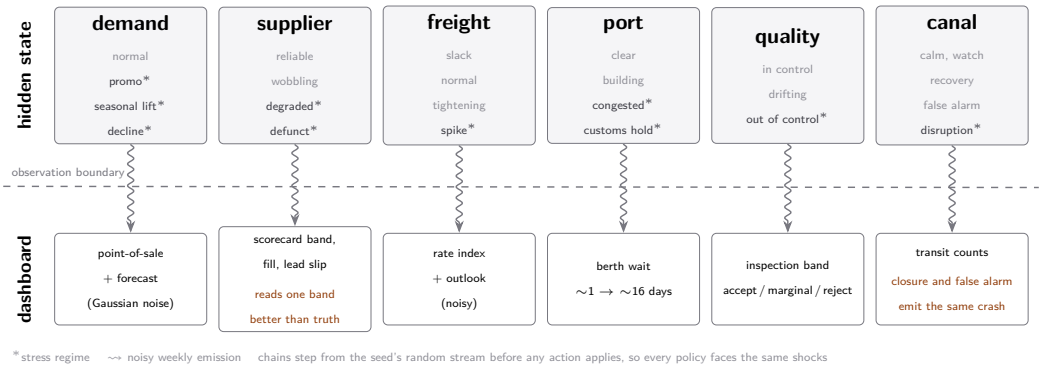

Figure 2: The hidden world behind the dashboard. The hidden state is six independent Markov chains; starred regimes are the stress regimes that define episodes in Section 4. Each chain emits one noisy symptom stream into the weekly dashboard, and those streams are all the agent (and the oracle) ever sees. Two symptoms mislead by construction (orange): the supplier scorecard reads one severity band better than the truth, and the canal's transit counts crash identically for a real closure and a false alarm.

3.2 The hidden world

The environment is a factored POMDP. Its hidden state decomposes into six independent factor processes — demand regime, supplier health, freight market, port congestion, batch quality, and canal status — each a small Markov chain that moves between a baseline and one or more stress regimes (Figure 2). The agent never observes a factor's state; each factor emits only noisy symptoms into the dashboard. Port congestion, for example, is a four-state chain (clear, building, congested, customs hold); when congested it inflates the observed berth wait from $\sim$1 day to $\sim$16 (plus Gaussian noise) and holds each arriving shipment an extra week.

Two symptoms mislead by construction. A degrading supplier ships late and short while its scorecard — deliberately — reads one severity level better than the truth. And the canal emits the same transit-count crash for a false alarm as for the first week of a real closure, so a Bayesian observer *should* be uncertain there.

Episodes are generated from a seed as a fixed event tape independent of the agent's actions: the factor chains step from the seed's random stream before any action is applied, so any two policies on the same seed face the identical sequence of shocks. This action-independence is what makes the reference policies of Section 4 directly comparable to any agent run. Full cost constants, caps, lead times, and per-factor transition parameters are in Appendix A.

3.3 Seeds and stress profiles

The benchmark fixes twenty seeds, selected from a 200-seed pool by replaying each tape and classifying its true stress profile:

- ISOLATED (6 seeds) — short, non-overlapping single-factor shocks;
- PERSISTENT (7 seeds) — a multi-week port blockage overlapping a demand rise. Here the intuitively appealing response, freezing orders until the blockage clears, is the costly one: orders placed *through* the blockage arrive as it lifts;
- COMPOUND (7 seeds) — pileups of three or more concurrent factor episodes.

Group membership is determined by the tape alone, before any agent is run, making the structure of hidden stress an independent variable of the benchmark.

3.4 Agent prompt

Agents run under a single *rules-only* prompt: it specifies the mechanics of the task (costs, lead times, levers, output format) and gives a qualitative description of the world's factors, but contains

no strategy advice, no playbook, and no worked examples. The benchmark thereby measures what a capable agent brings to the task, not what the prompt teaches it. Coached and structured-belief prompt variants are future work (Section 6).

4 Experimental Setup

Reference policies. Every run is scored between two reference policies evaluated on the same seed. The floor is a textbook base-stock policy: order up to a fixed target $S = \mu L + z\sigma\sqrt{L}$ (mean demand μ, lead time L, safety factor z from the holding/stockout cost ratio) every week, always by the same route and supplier, reading no symptom at all. The upper reference is the Bayes-filter oracle of Figure 1. Because each factor is a small discrete chain, its posterior can be maintained *exactly*: one forward (predict–correct) Bayes filter per factor, updated each week on the same observation dictionary that is serialized into the LLM prompt. To act, the oracle samples $\sim$200 possible futures of all six factors from these posteriors, scores a small menu of candidate actions (order sizes, routes, each mitigation lever, gated on the relevant posterior risk) against every sampled future, and executes the cheapest candidate through the same action API the LLM uses. Its randomness is independent of the world's, and it never reads hidden state. One asymmetry is disclosed: its filters use the environment's true generative parameters (transition probabilities, regime means), which the LLM receives only as a qualitative description. We therefore interpret the oracle as Bayes-optimal given the true model. Because it is a stochastic policy, its per-seed reference value is the mean of 20 independent replications (standard error 0.3–1.6%); it is a fair reference rather than an optimum, and agents exceed it on 5 of 60 runs. Appendix B specifies both policies in full.

Metrics. Let C_{base}, C_{orc}, and C_{agent} denote total episode cost for the floor, the oracle reference, and the agent. The skill score

$$\text{skill} \;=\; \frac{C_{\text{base}} - C_{\text{agent}}}{C_{\text{base}} - C_{\text{orc}}} \tag{1}$$

is 0 at the symptom-blind floor and 1 at the oracle mean; values above 1 and below 0 both occur. A worked example from seed 1: base-stock costs \$9,101, the oracle mean is \$6,932, and Claude Sonnet 5 spent \$8,539, so its skill is $(9101 - 8539)/(9101 - 6932) = 0.26$ — it recovered about a quarter of what seeing the symptoms was worth on that tape. For belief-side metrics, a *stress episode* is a maximal run of consecutive weeks in which one factor is truly in a stress regime (114 episodes across the 20 tapes). Detection lag is the number of weeks from episode onset until the factor is first named in the agent's rationale (averaged over detected episodes; never-detected episodes are reported separately), and the knowing-doing rate is

$$\text{KD} \;=\; \Pr(\,\text{stockout in week } t \mid \text{a truly stressed factor is named in week } t\,), \tag{2}$$

estimated over all stress weeks of a group, where "stockout" means the stock available that week (inventory plus arrivals) fell short of realized demand. A high KD rate means correct diagnosis coexisting with empty shelves.

Belief grading. Whether a rationale names a factor is judged by a small LLM grader (gpt-5-mini, temperature 0) that sees only the rationale text and the fixed factor vocabulary, never the tape; labels are aligned to the week the rationale was written. Grader labels were audited in two rounds against raw traces; residual disagreements are borderline over-labels on calm weeks, which the episode-based metrics ignore. The rationale is stated belief: a lower bound on what the model knows, a point we return to in Section 6. Grading rules and validation details are in Appendix C.

Models. We evaluate Claude Sonnet 5, GPT-5.4, and DeepSeek-V4-Pro through an identical harness, one run per model–seed pair (60 episodes), all under the rules-only prompt of Section 3.

5 Results

Detection is uniform. All three models see the hidden world about equally well (Table 2, Figure 3a). Each detects 85–92% of the 114 stress episodes, with a mean lag of 0.24–0.35 weeks: when a factor enters a stress regime, its symptoms are typically named in the rationale within the same week. DeepSeek-V4-Pro, the weakest model on cost, has the *shortest* detection lag. Whatever separates these models on this task, it is not state estimation.

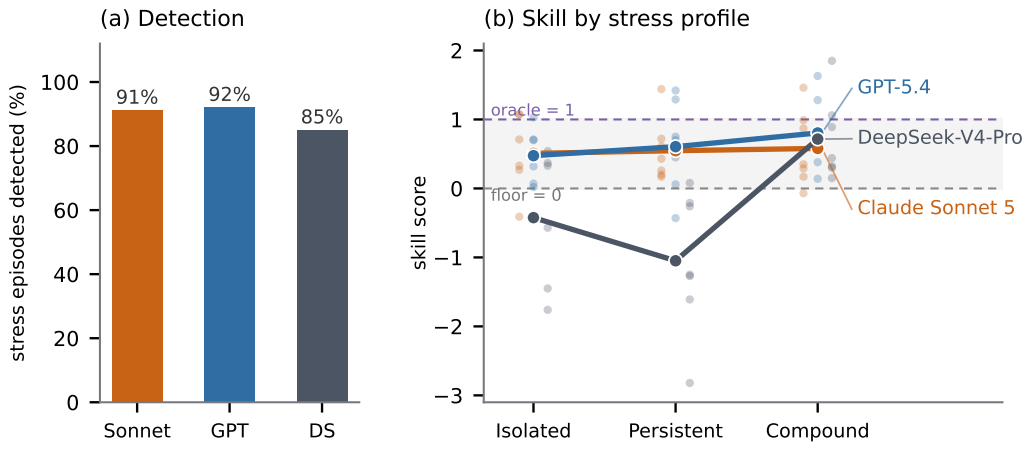

Figure 3: **Seeing is uniform; doing is not.** (a) All three models detect 85–92% of the 114 hidden-factor stress episodes, typically within a week of onset. (b) Group-mean skill by stress profile (lines), with per-seed scores as faint dots. Costs diverge across models and concentrate by profile: DeepSeek-V4-Pro falls below the symptom-blind floor on 9 of 20 seeds despite panel (a).

Table 1: Skill score by stress profile: group mean (median), 20 seeds per model, single run per model–seed pair. *neg* counts seeds scored below the base-stock floor.

Model	ISOLATED (n=6)	PERSISTENT (n=7)	COMPOUND (n=7)	All 20	neg
Claude Sonnet 5	0.51 (0.71)	0.55 (0.43)	0.58 (0.35)	0.55	2
GPT-5.4	0.48 (0.70)	0.61 (0.70)	0.80 (0.73)	0.64	1
DeepSeek-V4-Pro	−0.42 (0.33)	−1.05 (−1.25)	0.72 (0.44)	−0.24	9

Control is not. Skill scores diverge widely (Table 1, Figure 3b). Claude Sonnet 5 and GPT-5.4 recover 55% and 64% of the floor-to-oracle cost gap overall and stay above the base-stock floor on 18 and 19 of 20 seeds. DeepSeek-V4-Pro scores -0.24 overall and falls below the floor on 9 of 20 seeds: on nearly half the tapes, a symptom-blind base-stock rule with the same action menu would have been cheaper than acting on its (largely correct) diagnoses. Its COMPOUND mean of 0.72 is pulled up by a single outlier seed; the group median is 0.44. Individual seeds above 1 occur (5 of 60 runs) and are legitimate, since the oracle is Bayes-optimal in expectation but not per-tape. Even the best model leaves a third of the gap to a policy computed from identical observations, so the benchmark is far from saturated.

The gap concentrates on persistent stress. The knowing-doing rate (Table 2) shows where correct diagnosis fails to prevent empty shelves. For every model the rate peaks on PERSISTENT seeds, at 0.41–0.49, versus 0.06–0.30 on ISOLATED seeds: when a disruption lasts many weeks, a correctly diagnosed stress week still ends in stockout roughly half the time. Brief shocks are handled; sustained ones convert knowledge into cost. We flag the mechanism below as a hypothesis, not a demonstrated cause: the stockout in a given week can stem from an ordering decision made weeks earlier.

Two failure modes. Reading the traces suggests the models fail in opposite directions; full excerpts are in Appendix D. Claude Sonnet 5 on seed 1 (PERSISTENT; port congested weeks 17–24 while demand lifts) *freezes*: it diagnoses the congestion in week 16 and skips its sea order at once; the orders it does place are held up behind the blocked port, and through weeks 22–24, at zero inventory, it stops ordering by sea entirely (Figure 4 in the appendix plots the full episode). By week 22 it has held zero inventory for two weeks, yet writes "I already have a very large pipeline: 104 units in transit … ordering more now would only add to an already oversized pipeline … given berth_wait is still elevated (15 days)" and orders nothing, counting units trapped behind the congestion it itself diagnosed as usable cover. It ends with four consecutive zero-inventory weeks costing \$660–\$776 each, while the oracle keeps placing modest orders that arrive after the congestion clears.

Table 2: Belief-side metrics. Detection is pooled over the 114 stress episodes per model; the knowing-doing rate is the fraction of correctly-diagnosed stress weeks that still ended in stockout, by group.

	Detection		Knowing-doing rate			
Model	missed	mean lag (wk)	ISOL.	PERSIST.	COMP.	all
Claude Sonnet 5	10/114	0.35	0.30	0.49	0.27	0.37
GPT-5.4	9/114	0.30	0.06	0.41	0.29	0.31
DeepSeek-V4-Pro	17/114	0.24	0.14	0.49	0.34	0.37

DeepSeek-V4-Pro on seed 25 (ISOLATED; a two-week canal closure) *flails*: within two weeks of a correct diagnosis it locks freight rates for four weeks, air-expedites units, and stacks 80 units of orders against $\sim$20-per-week demand, on top of 78 already in the pipeline. The closure ends in week 5; the fees and holding costs it bought persist, and the episode finishes at skill -1.45 on one of the easiest tapes. Neither trace contains a detection error.

6 Discussion and Limitations

The results support one claim: on this task, frontier-model failures are concentrated in control, not state estimation, and the fair oracle makes that split measurable. A privileged oracle could not have licensed it, because any shortfall could be attributed to the information gap rather than to policy.

Stated belief is a lower bound. Detection is measured from the rationale the model writes, not from its internal state; a model may track a factor it never mentions. This biases detection down, which strengthens the seeing-vs-doing contrast, but the knowing-doing rate inherits the grader's labels. Two audits (a stratified manual read and an independent second grader) agree with the labels on the metric-relevant component at $\sim$90%; residual disagreement sits on calm-week over-labels the episode-based metrics exclude.

Single runs and provider nondeterminism. Each model–seed cell is one run; provider-side nondeterminism is unquantified. Group means pool 6–7 seeds, and the cross-model contrasts (a 0.9-point skill spread against a 0.07 detection spread) are far larger than plausible single-run noise, but per-seed values should be read as samples, not estimates. The oracle reference is a 20-replication mean (standard error 0.3–1.6% of the reference cost).

The oracle knows the physics. The oracle receives the identical observation stream, never the hidden state, but its filters use the true generative parameters (regime means, transition probabilities), while the LLM sees only a qualitative world description. Skill $= 1$ therefore means "Bayes-optimal given the true model", a demanding but not information-free reference. Scores above 1 are possible and observed.

Scope. One domain, one SKU, one prompt arm, three models. The tapes are generated from published seeds rather than scraped, so they cannot occur in training data, but conclusions about *why* the gap concentrates on persistent stress await intervention: prompt arms that coach explicit belief maintenance, repeated runs per cell, a clairvoyant ceiling to bound the oracle's optimality gap, and a broader model set. These are the immediate next steps for the benchmark.

A World and Cost Parameters

All values below are read directly from the environment source; they fully specify the weekly cost function and the six hidden factor processes of Section 3.

Costs. Each week's cost is the sum of the applicable components:

Component	Charge	When
Sea shipping	\$4.00 (Suez) / \$6.00 (Cape) per unit $\times$ freight mult.	at dispatch
Supplier price delta	qualified $+$\$1.00, spot $-$\$1.50, backup $+$\$0.30 per unit	at dispatch
Holding	\$1.00 per unit-week, on hand and in transit	weekly
Stockout	\$20.00 per unit of unmet demand (lost sales)	weekly
Port demurrage	\$2.00 per unit held at a blocked port	while blocked
Quality rework	\$15.00 per defective unit landed	at arrival
Diversion surcharge	\$2.00 per unit forced from Suez around the Cape	at diversion
Crisis back-order	\$60.00 per unit a supplier shorts during a canal event	coupled
Air expedite	\$15.00 per unit, at most 20 units/week	on use
Briefing / audit / inspection	\$30 / \$25 / \$40 flat per call	on use
Dual-sourcing overhead	\$4.00 per week while two or more contracts are live	weekly

Orders are capped at 100 units per week. Sea lead times are 3 weeks via Suez and 4 via the Cape; air freight lands the following week. The implied newsvendor critical ratio is $20/21 \approx 0.95$, which fixes the base-stock safety factor $z \approx 1.67$ used by the floor policy.

Factor processes. Each factor is an independent Markov chain stepped from the seed's random stream before any action applies. Starred regimes are the stress regimes that define episodes in Section 4.

Factor	Regimes	Key dynamics and observations
Demand	normal, promo spike*, seasonal lift*, structural decline*	onsets 0.03/0.015/0.005 per week; promo lasts 4 weeks, seasonal persists 0.85 (cap 8), decline persists 0.97. Weekly means 20/26/30/14 units; observed point-of-sale and forecast carry Gaussian noise (sd 4 and 6).
Supplier	reliable, wobbling, degraded*, defunct*	onset 0.10, wobbling$\rightarrow$degraded 0.45; degraded recovers within 4 weeks or dies (0.06). Scorecard band reads one severity level better than the truth; lead-time slip and fill fraction are noisy.
Freight	slack, normal, tightening, spike*	tightening onset 0.06, tightening$\rightarrow$spike 0.18, spike persists 0.80 (cap 6). Rate multipliers 0.7/1.0/1.8/4.0; index and outlook noisy.
Port	clear, building, congested*, customs hold*	building onset 0.06, building$\rightarrow$congested 0.30, congested persists 0.85 (cap 8); customs holds last about a week. Berth wait rises from $\sim$1 to $\sim$16 days; a blocked port delays every arrival one week.
Quality	in control, drifting, out of control*	drift onset 0.04; the drift-to-failure hazard grows with age ($0.05 + 0.04 \cdot$ age). Defect fractions 0.1%/2%/6%; observed only as a noisy accept/marginal/reject inspection band.
Canal	calm, watch, disruption*, recovery, false alarm	calm$\rightarrow$watch 0.08; watch resolves to a real closure (0.50) or a false alarm (0.20). Transit counts are exact but the first closure week and a false alarm emit the identical "crash" reading. A closed canal queues Suez ships, then diverts them ($+$3 weeks, Cape rate, surcharge).

B Reference Policy Details

Base-stock floor. Order-up-to level $S = \mu L + z\sigma\sqrt{L}$ with $\mu = 20$, $\sigma = 4$, $L = 4$ (3-week Suez lead plus one review week), $z = \Phi^{-1}(20/21) \approx 1.67$. Every week it orders $\max(0, S -$ inventory position$)$, always via Suez from the same supplier, and never touches a lever or reads a symptom.

Oracle decision loop. Each week the oracle (i) updates one exact forward Bayes filter per factor on the shared observation dictionary; (ii) samples $\sim$200 joint futures of all six factors from the filter posteriors, rolled forward with the true transition tables; (iii) scores a small candidate menu against

every sampled future using a fast surrogate of the cost function, holding future weeks to a base-stock continuation; and (iv) executes the cheapest candidate through the same action API the LLM uses. The candidate menu contains order quantities $\{0$, four weeks, six weeks of expected demand$\}$, both routes, the contracted suppliers, and each mitigation lever gated on posterior risk (freight lock if $\Pr(\text{tightening or spike}) > 0.3$; air expedite if port risk > 0.3; inspection if quality risk > 0.5 and a batch lands next week). The same sampled futures are reused across candidates (common random numbers). The oracle never buys briefings or audits; the LLM may.

C Belief Grading Details

The grader receives only the week's rationale text and the fixed six-name factor vocabulary, and returns strict JSON. Its instructions require a factor to be labeled only if the text claims it is a problem *that week*: ruled-out or benign mentions do not count, factual symptom reports do count, and precautions without a current problem do not count. Mean detection lag is computed over detected episodes only; naming a factor before its episode starts never counts as detection. In the knowing-doing rate, "stockout" means the week's available stock (inventory plus arrivals) fell short of realized demand. Validation: on a stratified 30-row manual read, all 10 sampled hit-class labels were correct and 9 of 10 sampled misses were genuine (the tenth ambiguous, in the direction that understates detection); an independent second grader (Claude Haiku 4.5, same prompt, 100 sampled weeks) agreed 43% on exact label sets but 89% on the metric-relevant quantity, the intersection of labeled and truly stressed factors. Calm-week over-labels, where graders disagree most, are excluded from every reported metric by construction.

The grader's full instructions are reproduced below; `{rationale}` is replaced by the week's rationale text.

```
You will be given one week's replenishment-decision rationale from a supply-
chain manager. The fixed factor vocabulary is exactly these 6 names:
disruption, freight, port, quality, supplier, demand.

Return STRICT JSON only, of the form: {"factors_named": [subset of the 6
names]}

Include a factor ONLY if the rationale asserts it is CURRENTLY
active/problematic this week. Go through the 6 factors one by one and ask:
does the text CLAIM this is a problem right now? Three rules, each with a real
example:

1. RULED-OUT or BENIGN mentions do NOT count. "freight remains cheap" = no
   freight. "the weak fill looks noisy rather than confirmed deterioration" =
   no supplier. "lanes normal" = no disruption. Naming a factor to say it is
   fine is the OPPOSITE of naming it as a problem.
2. FACTUAL symptom reports DO count even without editorializing. "AQL reject
   -- inspected batch" = quality. "port congestion (berth_wait 16) delayed
   arrivals longer than expected" = port. The manager asserts a problem exists
   by reporting its symptom as the cause of something.
3. PRECAUTIONS without a current problem do NOT count. "I'll keep using the
   freight lock" alone = no freight. "avoiding spot until it stabilizes" after
   calling the data noise = no supplier. But acting BECAUSE of an asserted
   current problem ("air-expediting because the port is holding arrivals") =
   count that factor.

Worked example -- rationale: "Freight lock still active (1.19x) shielding us
from the 179 spot index. AQL came back marginal causing a rework charge. Spot
remains slipping (86 OTIF) so still avoiding it." -> {"factors_named":
["freight", "quality", "supplier"]} (index 179 asserted high; AQL/rework is a
current quality symptom; "spot remains slipping" is a current supplier claim.)

Rationale:
"""{rationale}"""

JSON:
```

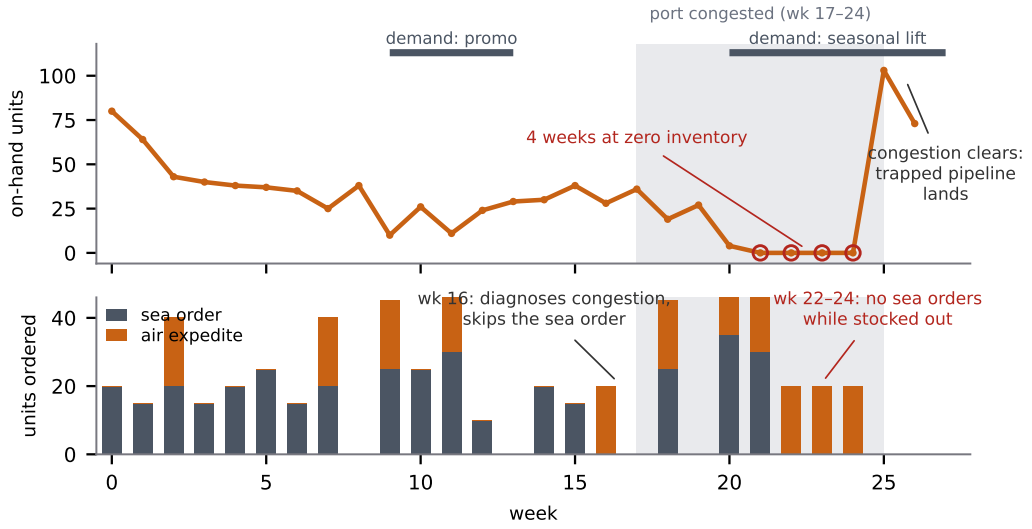

Figure 4: **The freeze, week by week (Claude Sonnet 5, seed 1).** Top: on-hand inventory; the shaded band is the true port-congestion window (weeks 17–24) and the rugs above mark the true demand windows, none of which the agent observes directly. Bottom: units ordered by sea and by air each week. The agent diagnoses the congestion in week 16, its sea orders pile up behind the blocked port, and in weeks 22–24 it stops ordering by sea while holding zero inventory — reasoning that the trapped pipeline is ample cover. When the congestion clears in week 25, the trapped orders land at once. Every series is parsed from the raw episode trace.

D Seed Timelines and Trace Excerpts

The table below gives the ground-truth stress windows for the nine originally released seeds, obtained by replaying each tape with no actions (the tape is action-independent). Each entry reads *factor: regime, weeks active*; factors are the six of Section 3. Windows joined by "+" overlap in time — the defining feature of the PERSISTENT and COMPOUND groups. Timelines for the eleven expansion seeds ship with the released tapes.

Seed	Group	True stress windows (weeks active)
157	ISOLATED	canal closed 12–13; quality out of control 23–24; freight spike 25–26
25	ISOLATED	canal closed 4–5; port customs holds 6–7, 12, 21, 26; supplier degraded wk 20
112	ISOLATED	demand shifts only: decline 13–14, seasonal lift 20–24
1	PERSISTENT	port congested 17–24 + demand seasonal lift 20–26; demand promo 9–12
172	PERSISTENT	port congested 5–11 + demand promo 9–12; supplier degraded 24–26
170	PERSISTENT	freight spike 3–8 + port congested 5–12; demand promos 2–5, 15–18; freight spike 24–26
21	COMPOUND	demand seasonal lift 10–17; then weeks 21–26: canal closed + quality out of control + supplier degraded at once
143	COMPOUND	freight spike 5–10 running into port congestion 10–17; supplier degraded 2–5; canal closed wks 15, 25
99	COMPOUND	the marathon tape: three port-congestion episodes, two canal closures, three demand-regime changes

Exhibit A: freezing (Claude Sonnet 5, seed 1). Week 16, on diagnosing the congestion: "Port congestion has spiked severely (berth_wait 10 days…) … I'll skip placing a new sea order this week … and reassess next week once conditions hopefully ease." Week 22, with zero on-hand inventory and a stockout in progress: "Despite the current 0 on-hand inventory, I already have a very large pipeline: 104 units in transit (arriving weeks 23–25) … roughly 4–5 weeks of cover

... ordering more now would only add to an already oversized pipeline, incurring extra holding/in-transit costs and demurrage risk given berth_wait is still elevated (15 days)." Weeks 21–24 all close at zero inventory ($660–$776 per week).

Exhibit B: flailing (DeepSeek-V4-Pro, seed 25). Week 4: "Week 4: Disruption confirmed — Suez counts crashed to 28/25 ... Cape is now the only safe route. Locked freight at 1.02x for 4 weeks. Ordering 40 via Cape/spot." Week 6, correctly closing the diagnosis: "Week 6: Suez reopening — counts back to 56/56." By then the freight lock, air expedite, and stacked orders are sunk; weekly costs run $610–$814 through week 8, and the episode ends at $7,950 total cost, skill -1.45.

E Agent Interface: Prompt and Observation Format

Weekly observation. The observation dictionary below is serialized into the agent's context each week, exactly as produced by the environment (a presentation-manifest key used only by the web frontend is omitted). This example is week 10 of seed 1 under the base-stock policy: the demand promo is active (point-of-sale 29 against a baseline of 20), the lane is calm, and the symptom-blind policy has just taken a small stockout.


```
{
 "week": 10,
 "inventory": 0,
 "on_order": 66,
 "inventory_position": 66,
 "arrived": 27,
 "components": {
  "unit": {
   "on_hand": 0,
   "on_order": 66,
   "inventory_position": 66,
   "arrived": 27
  }
 },
 "served": {
  "unit": 27
 },
 "pipeline": [
  {
   "qty": 23,
   "route": "suez",
   "supplier": "spot",
   "component": "unit",
   "dispatched_week": 8,
   "status": "at_sea",
   "eta": 11
  },
  {
   "qty": 15,
   "route": "suez",
   "supplier": "spot",
   "component": "unit",
   "dispatched_week": 9,
   "status": "at_sea",
   "eta": 12
  },
  {
   "qty": 28,
   "route": "suez",
   "supplier": "spot",
   "component": "unit",
   "dispatched_week": 10,
   "status": "at_sea",
   "eta": 13
  }
 ],
```

```
"cost_breakdown": {
 "shipping": 80.192,
 "surcharge": 0.0,
 "holding": 0.0,
 "in_transit": 66.0,
 "stockout": 40.0,
 "couple": 0.0,
 "demurrage": 0.0,
 "rework": 0.0
},
"contracts": [
 {
  "supplier": "spot",
  "start_week": 0,
  "end_week": null,
  "unit_price": 4.0,
  "otif_floor": 85,
  "break_fee": 10.0
 }
],
"contract_open": [],
"term_menu": [
 "short",
 "long",
 "strict",
 "lenient"
],
"suez_count": 70,
"bab_count": 70,
"cape_count": 60,
"bulletin": "Shipping lanes normal. Suez Canal and Bab-el-Mandeb transits at
  seasonal levels; Cape routing remains a niche choice.",
"suppliers": [
 {
  "id": "qualified",
  "otif": 99,
  "lead_days": 14,
  "band": "ontime",
  "onboard_lead": 0,
  "unit_premium": 1.0,
  "unit_delta": 1.0
 },
 {
  "id": "spot",
  "otif": 98,
  "lead_days": 14,
  "band": "ontime",
  "onboard_lead": 0,
  "realized_lead_slip": 0.0,
  "unit_discount": 1.5,
  "unit_delta": -1.5
 },
 {
  "id": "backup",
  "otif": 95,
  "lead_days": 16,
  "band": "ontime",
  "onboard_lead": 1,
  "unit_delta": 0.3
 }
],
"pos_units": 29,
"demand_forecast": 28,
"freight_index": 109,
"freight_outlook": 85,
"berth_wait": 0,
"wait_outlook": 8,
"aql_result": "accept",
```

```
 "realized_fill": 1.0
}
```

System prompt (FAITHFUL arm). The complete system prompt used for all 60 runs is reproduced verbatim below. It specifies mechanics, costs, tool semantics, and the latent structure of each factor, and contains no strategy advice; its inclusion substantiates the rules-only claim of Section 3.

```
You run the import replenishment desk for a European importer on the Asia-
Europe shipping lane. You will run it for 26 weeks. Your one objective is to
MINIMIZE TOTAL COST over the whole 26-week horizon -- not any single week.

THE WEEK
Each week you see the current situation, then make exactly one decision by
calling place_order. The world advances only when you call place_order. You
start with 80 units on hand. Demand is roughly 20 units a week (it drifts --
see DEMAND below), served from on-hand inventory; unmet demand is a stockout.

YOUR LEVERS (these are the only actions; mirror them exactly)
- place_order(rationale, qty, route, supplier, contract_action,
  contract_supplier, contract_terms): one weekly decision that may place an
  order, manage a contract, or both.
  - rationale (REQUIRED, every week): a few sentences working through THIS
week's decision and why this qty/route/supplier (and any contract). The world
does not advance without it.
  - qty: any whole number of units to order this week, from 0 up to 100. 0 =
order nothing (no route/supplier needed). There is no fixed menu.
  - route "suez" or "cape" (required if qty > 0):
    - "suez": base 4/unit, faster (~3 weeks), but the Suez/Red Sea corridor
can be disrupted -- a ship caught at the canal during a disruption waits, then
diverts around the Cape (arriving much later and billed the Cape price
difference).
    - "cape": base 6/unit, slower (~4 weeks), but it bypasses the Suez
corridor and is reliable.
  - supplier "qualified", "spot", or "backup" (required if qty > 0). You may
only source a supplier you hold a LIVE contract with -- see CONTRACTS and the
`contracts` / `contract_open` keys in the report.
  - to manage a contract THIS week, set contract_action ("sign", "switch",
"renew", or "lapse"), contract_supplier, and (for sign/switch/renew)
contract_terms ("short", "long", "strict", "lenient"). The contract resolves
BEFORE the order, so you can sign a supplier and source it in the same call.
Use qty 0 to manage a contract without ordering.
- buy_briefing(): pay 30 for a one-line analyst assessment of THIS week's LANE
  state (the Suez corridor), before you order. Optional.
- buy_audit(): pay 25 for a direct read of your spot supplier's current
  reliability state, before you order. Optional.
- lock_freight(weeks): forward-buy the freight rate -- FIX this week's freight
  cost multiplier for the next `weeks` weeks. While locked you pay the locked
  rate regardless of the spot index: it shields you from a rate spike, but you
  forgo a drop, and an unused week still burns the window. A within-week
  action (it does NOT advance the week); lock, then place_order in the same
  week to ship at the locked rate. The active lock shows as `freight_lock`
  (rate + weeks_left) in the report.
- expedite_air(qty): fly units in on a fast air lane that BYPASSES the
  destination port -- they land in your inventory NEXT week regardless of port
  congestion, at 15/unit, capped at 20 units a week. A within-week action (it
  does NOT advance the week); expedite, then place_order in the same week.
- inspect_batch(): pay 40 to run an incoming inspection on THIS week's
  arriving batch -- it sorts and reworks the defects, recovering about 70% of
  them before they stock (fewer units lost to defects, less rework). A within-
  week action (it does NOT advance the week); inspect, then place_order in the
  same week.

SUPPLIERS (who you buy from -- pick per order)
- qualified (the incumbent): reliable (99% OTIF), but dearest -- it adds
  1.0/unit over the route base (Suez 5/unit, Cape 7/unit). Always ships your
  full quantity. Evergreen contract. In this task you are NOT contracted to
  qualified at the start -- sign it to migrate off spot when you judge spot
```

```
   has turned.
- spot (YOUR STARTING INCUMBENT: you begin already contracted to it and source
   it by default): cheapest -- 1.5/unit BELOW the route base (Suez 2.5/unit,
   Cape 4.5/unit) -- but its reliability DRIFTS and you cannot see it directly.
   A healthy spot ships your full qty; a wobbling one ships only about HALF; a
   degraded one ships NOTHING; and it can go DEFUNCT (fail for good, gone for
   the rest of the horizon). Its OTIF scorecard (ontime / slipping / failing /
   defunct) is the contracted on-time metric; you also see your realized
   experience with spot -- realized_fill (how much of an order actually shipped
   when you sourced it) and realized_lead_slip (its reported lead-time this
   week), both noisy week to week. buy_audit gives a direct read of its current
   state. "slipping" is ambiguous -- it is either a wobble that recovers or the
   first week of a real failure; the following weeks tell you which. AND a spot
   shortfall DURING a lane disruption is back-ordered at a crisis rate about 3x
   a normal stockout.
- backup (a second qualified source): reliable (95% OTIF), a small premium
   (+0.3/unit over the route base), but it needs 1 week of onboarding before
   its FIRST order can ship.

 CONTRACTS (the gate on sourcing)
- You can only source a supplier you currently hold a live contract with. The
   report shows your `contracts`, `contract_open` (contracts that have expired
   or whose supplier died -- these need renewing), and `term_menu`.
- contract_action "sign"/"switch"/"renew" opens a fresh contract on the chosen
   supplier with the chosen terms; "lapse" drops an open contract (surrender).
   A contract auto-opens when it expires or its supplier dies, and qualified's
   contract is evergreen.
- terms menu (the locked unit price is set off the Suez base, 4/unit):
     - "short": 4 weeks, ~3% cheaper, easy to exit.
     - "long": 12 weeks, ~6% dearer (a price-lock), hard to exit.
     - "strict": 8 weeks, high OTIF floor (the supplier owes you on a slip).
     - "lenient": 8 weeks, ~5% cheaper, no real penalty -- you eat the risk.
- Carrying 2 or more live contracts costs 4/week (dual-source overhead).

 COSTS (every number is real; weigh them)
- shipping: the route base, adjusted for the supplier (above), then scaled by
   the freight index (see FREIGHT), paid when you order.
- holding: 1 per unit per week -- on inventory ON HAND and IN TRANSIT (capital
   on the water still costs you).
- stockout: 20 per unit of unmet demand in a week.
- surcharge: a Suez ship diverted around the Cape is billed the Cape-vs-Suez
   difference.
- demurrage: 2 per held unit per week when the destination port holds your
   arrivals (see PORT).
- air: 15 per unit when you expedite_air to fly units past a jammed port (see
   PORT).
- rework: 15 per defective unit when quality is off (see QUALITY).
- inspect: 40 when you inspect_batch to sort a bad arriving batch (see
   QUALITY).
- briefing: 30 each; dual-source overhead: 4/week.

 WHAT YOU SEE EACH WEEK (your only signals; the latent ones are NOISY)
- week, inventory, arrived, and pipeline (your in-flight shipments with
   estimated arrival weeks).
- inventory_position and on_order: on_order is the total units already ordered
   but not yet arrived (your pipeline); inventory_position = inventory on hand
   + on_order.
- LANE: suez_count, bab_count, cape_count (ships that transited the Suez
   Canal, the Bab-el-Mandeb strait, and the Cape this week) plus a trade-press
   bulletin.
- SUPPLIERS: an OTIF scorecard per supplier (band + on-time % + quoted lead).
   For spot you also see realized_fill (actual vs ordered, when you sourced it)
   and realized_lead_slip (its reported lead behaviour this week).
- DEMAND: pos_units (what actually sold this week) and demand_forecast (a
   forward read). Both noisy. Underlying demand drifts between normal, short
   promo spikes, sustained seasonal lifts, and a sticky structural decline.
- FREIGHT: freight_index (this week's spot-rate level; ~100 is normal) and
   freight_outlook (a noisier forward read). Your shipping cost is scaled by
```

```
   the index the week you order. The underlying rate regime drifts between
   slack (cheap), normal, tightening, and a costly spike.
- PORT: berth_wait (days) and wait_outlook. When the destination port congests
   or a customs hold lands, your arrivals are held a week and accrue demurrage.
- QUALITY: aql_result (accept / marginal / reject -- incoming inspection).
   When the supplier's process drifts out of control, a fraction of your
   arrivals are defective: they do not stock and they cost rework.
- cost_breakdown: what last week cost you, by category.

THE LANE STRUCTURE (told to you plainly -- use it)
There is a disruption process on the Suez corridor that you cannot observe
directly. It builds up, may or may not break into a real disruption, and if it
does, the disruption is either short (clears within a couple of weeks) or long
(drags on for many weeks). You only learn which by tracking how the weekly
counts, the bulletin, and your shipment ETAs evolve over time.

When trouble first breaks, a genuine disruption and a false alarm look
IDENTICAL for one week -- the counts drop the same way whether it is nothing
or the first week of a real disruption. The ambiguity resolves the FOLLOWING
week: a false alarm snaps back to normal; a real disruption stays down, and
its counts then reveal whether it is the short or the long kind.

THE LOOP (you own it)
Run the full episode yourself. Each week: read the latest situation report (it
arrives with the kickoff and with every order you place), optionally use the
within-week levers, then call place_order exactly once -- with a written
'rationale' for that week's decision. Placing the order advances the world to
the next week and returns the new report. Keep going week after week until
place_order tells you the episode is done. Do NOT ask the human anything. Do
NOT stop early. Every week's reasoning goes in the place_order 'rationale' so
your thinking is always visible.
```